

Einführung in die Programmierung

Am Beispiel **Kotlin** 🌴

Drücke  oder  um weiterzublättern

Fahrplan — 2 Tage

📖 **Teil 1** — Was ist Programmierung?

📖 **Teil 2** — Kotlin Grundlagen

🎤 **Live Demo** — Wir programmieren zusammen

👉 **Hands-on** — FizzBuzz Challenge

📖 **Teil 3** — Objektorientierung

👉 **Hands-on** — Steckbrief

🐾 **Coding Kata** — Digital Pet



Theorie · Demo · Selber bauen

Wir wechseln zwischen **Theorie**, **Live-Demo** und **Hands-on**.

Was lernen wir — und was nicht?

Diese 2 Tage

-  Grundbausteine: Variablen, Schleifen, `if / else`
-  Daten in Klassen organisieren
-  Ein kleines, vollständiges Programm bauen
-  Wie ein Programmierer **denken**

Das schaffen wir nicht (heute)

-  Eigene Android-App veröffentlichen
-  3D-Spiele bauen
-  Datenbanken, Web, Netzwerk
-  Künstliche Intelligenz

Aber: Mit den Grundlagen von heute könnt ihr **jede** dieser Sachen später lernen — die Konzepte sind in allen Sprachen ähnlich.

Bestandteile einer Programmiersprache

Fast jede Programmiersprache hat dieselben Grundbausteine:

Werte & Typen

Zahlen, Text, Wahrheitswerte

→ ``42`, `"Hallo"`, `true``

Operatoren

Werte verknüpfen und vergleichen

→ ``+`, `-`, `==`, `>``

Funktionen

Wiederverwendbare Code-Bausteine

→ ``fun begruessen() { ... }``

Variablen

Werte mit einem Namen speichern

→ ``var x = 5``

Kontrollfluss

Entscheidungen und Wiederholungen

→ ``if`, `for`, `while``

Klassen & Objekte

Daten + zugehörige Funktionen bündeln

→ ``class Pet(val name: String)``

 All das schauen wir uns in Kotlin an.

Null-Safety · Coroutines

📦 FUNDAMENTALS

Variablen · `if`
Schleifen · Funktionen
Klassen

← unsere 2 Tage

So sieht Kotlin aus

Ein kleines Programm — wir lesen es **zusammen**.

```
1 fun main() {  
2     val spieler: String = "Mia"  
3     var punkte: Int = 4  
4     var bonus: Int = 3  
5  
6     punkte = punkte * 2  
7     bonus = bonus + punkte  
8     punkte = bonus - 5  
9  
10    println("Hallo $spieler!")  
11    println("Punkte: $punkte · Bonus: $bonus")  
12  
13    if (punkte >= 20) {  
14        println("Legendär! 🏆")  
15    } else if (punkte >= 10) {  
16        println("Stark! 💪")  
17    } else {  
18        println("Übung macht den Meister.")  
19    }  
20 }
```

 `fun main() { ... }` — eine **Funktion**: Name, runde Klammern für Argumente, geschweifte für den Code.

 `val = fest · Typ` `String` (Text).

 `var = änderbar · Typ` `Int` (Zahl).

+ Zuweisungs-Kette — jede Zeile rechnet mit den **neuen** Werten.

 `println(...)` — Ausgabe; `$spieler` setzt den Wert ein.

 `if / else if / else` — wählt **einen** der Zweige.

 **Mini-Rätsel**: Was steht am Ende auf dem Bildschirm?



Teil 1

Was ist Programmierung?

Was ist Programmierung?

Programmieren = eine **menschlich lesbare Schicht**, um dem Computer Anweisungen zu geben.



Der Computer versteht nur **Nullen und Einsen**.

Programmiersprachen sind die **Brücke** zwischen unserer Idee und der Maschine.

Wo steckt überall Programmierung drin?

Bereich	Beispiele
 Apps	Instagram, WhatsApp, TikTok
 Websites	YouTube, Google, Wikipedia
 Spiele	Minecraft, Fortnite, FIFA
 Geräte	Roboter, Drohnen, Smartwatches
 Autos	Autopilot, Einparkhilfe
 KI	ChatGPT, Bilderkennung, Sprachassistenten

Praktisch alles, was einen Bildschirm hat (und vieles ohne) wird programmiert.

Welche Programmiersprachen gibt es?

Hunderte — jede hat ihren Einsatzbereich:

Sprache	Typischer Einsatz
Python	KI, Data Science, Automatisierung
JavaScript	Websites, Web-Apps
Java	Große Unternehmenssoftware, Android
Kotlin	Android-Apps, Server, Multiplatform
Swift	iPhone-Apps
C / C++	Betriebssysteme, Spiele-Engines
C#	Spiele (Unity), Windows-Software

Was ist Kotlin?

- Moderne Sprache von **JetBrains** (seit 2016 stabil)
- Von **Google** als bevorzugte Sprache für Android empfohlen
- Läuft auf der **JVM** (Java Virtual Machine)
- Klare, lesbare Syntax — weniger Code, gleiches Ergebnis
- Name kommt von der **Insel Kotlin** bei St. Petersburg 🌴

Java

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hallo Welt!");  
    }  
}
```

Kotlin

```
fun main() {  
    println("Hallo Welt!")  
}
```

Kein Semikolon, keine Klassen-Hülle nötig — einfach loslegen.

Warum Kotlin für uns?

Zum Lernen

- Einfach zu lesen und schreiben
- Sofort im Browser testbar → kein Setup
- Fehler werden früh erkannt

In der echten Welt

- Offizielle Sprache für Android
- Google, Netflix, Amazon, Uber nutzen Kotlin
- Wächst stark — gute Jobchancen

👉 Echte Case Studies: kotlinlang.org/case-studies

Wann eher etwas Anderes?

Aufgabe	Sprache
Websites	JavaScript
KI	Python
iPhone-Apps	Swift
3D-Spiele	C# / C++

Unser Werkzeug: Kotlin Playground

Direkt im Browser — kein Setup nötig:



play.kotlinlang.org

```
fun main() {  
    println("Hallo Welt!")  
}
```

Jedes Kotlin-Programm startet in der `main()` -Funktion.

Alles zwischen `{` und `}` wird ausgeführt.



Teil 2

Kotlin Grundlagen

Was ist eine Funktion?

Eine **Funktion** ist ein **Mini-Programm** mit Namen.

- Du kannst sie **immer wieder aufrufen**
- Sie macht **eine bestimmte Sache**
- Sie hat einen **Namen**, der beschreibt was sie tut

Wie eine Mikrowelle: Knopf drücken (aufrufen) → sie macht ihr Ding.

Aufbau in Kotlin

```
fun begruessen() {  
    println("Hallo!")  
    println("Wie geht's?")  
}
```

Teil	Bedeutung
fun	„Hier kommt eine Funktion“
begruessen	Name der Funktion
()	(Eingaben — später)
{ ... }	Was sie tut

Aufrufen:

```
begruessen() // führt den Code drin aus
```

main() — wo alles startet

Jedes Kotlin-Programm hat **eine besondere Funktion**: `main()` .

```
fun main() {  
    println("Hallo Welt!")  
}
```

Sie wird vom Computer **automatisch aufgerufen**, wenn das Programm startet.

👉 Alles was passieren soll, schreibst du **zwischen die { }** **von** `main` .

println — Ausgabe

Mit `println()` gibst du Werte auf der Konsole aus.

```
fun main() {  
    println("Hallo Welt!")    // Text  
    println(42)               // Zahl  
    println(3 + 4)            // Berechnung → 7  
}
```

`println` = **print line** → gibt eine Zeile aus.

Variablen — Werte speichern

Variablen = **beschriftete Boxen** für Werte.

```
var punkte = 0 // veränderbar
val name = "Kotlin" // fest
```

Keyword	Bedeutung	Änderbar?
<code>var</code>	variable	✅ Ja
<code>val</code>	value (Wert)	❌ Nein

```
punkte = 10 // ✅ geht (var)
name = "Java" // ❌ Fehler (val)
```

Faustregel: Schreibe so oft wie möglich `val` .

Nur `var` wenn der Wert sich wirklich ändern muss.

Datentypen — Was passt in die Box?

```
val alter: Int = 16           // Ganzzahl  
val groesse: Double = 1.75    // Kommazahl  
val name: String = "Alex"     // Text  
val cool: Boolean = true      // Wahr / Falsch
```

Typ	Was?	Beispiele
Int	Ganzzahl	0, 42, -7
Double	Kommazahl	3.14, 1.75
String	Text	"Hallo", "Kotlin"
Boolean	Wahr / Falsch	true, false

Kotlin erkennt den Typ meistens automatisch — man muss ihn oft nicht hinschreiben.

Wozu überhaupt Typen?

Ein Typ sagt: „In dieser Box darf nur DAS rein.“

```
val alter: Int = 16  
alter = "sechzehn" // ✖ Fehler schon beim Tippen!
```

-  **Sicherheit** — viele Fehler werden gefunden, **bevor** das Programm läuft
-  **Autovervollständigung** — die IDE weiß, was du mit dem Wert machen kannst
-  **Lesbarkeit** — man sieht sofort, was eine Variable bedeutet
-  **Geschwindigkeit** — der Computer weiß genau, wie viel Platz er braucht

Sprachen **ohne** Typen (z.B. Python, JavaScript) sind flexibler — finden Fehler aber oft erst beim Ausführen.

Typ-Inferenz — Kotlin denkt mit

In **Java** musst du den Typ immer hinschreiben:

```
int alter = 16;  
String name = "Alex";  
double pi = 3.14;  
boolean cool = true;
```

Doppelt gemoppelt: `int` steht da, obwohl `16` offensichtlich eine Ganzzahl ist.

In **Kotlin** reicht der Wert — der Typ wird **erkannt**:

```
val alter = 16           // → Int  
val name = "Alex"       // → String  
val pi = 3.14           // → Double  
val cool = true         // → Boolean
```

Der Compiler schaut sich den Wert rechts an und leitet den Typ **automatisch** ab — **Typ-Inferenz**.
Weniger tippen, gleiche Sicherheit.

Wann doch explizit?

Auch in Kotlin schreibst du den Typ manchmal **bewusst** hin:

1. Wenn der Typ nicht aus dem Wert hervorgeht

```
val temperatur: Double = 20    // sonst wäre es Int
```

2. Wenn du den Typ allgemeiner halten willst

```
val zahlen: List<Int> = listOf(1, 2, 3)
```

3. Bei Funktions-Parametern und Rückgaben — Pflicht!

```
fun begruessung(name: String): String {  
    return "Hallo $name"  
}
```

4. Wenn es die Lesbarkeit verbessert — z.B. bei langen Ausdrücken

Faustregel: **Variablen** meistens ohne Typ, **Funktionen** immer mit.



Quiz — `var` oder `val`? Welcher Typ?

`val` `name` = `"Lena"`

→ `String`

`var` `punkte` = `0`

→ `Int` (zählt hoch)

`val` `pi` = `3.14`

→ `Double`

`var` `eingeloggt` = `false`

→ `Boolean`

String Templates

Mit `$` baust du Variablen direkt in Text ein:

```
val name = "Alex"  
val alter = 16  
  
println("Ich heiÙe $name und bin $alter Jahre alt.")  
// → Ich heiÙe Alex und bin 16 Jahre alt.
```

Für **Berechnungen** → geschweifte Klammern `${...}` :

```
println("In 2 Jahren bin ich ${alter + 2}.")  
// → In 2 Jahren bin ich 18.
```


Mathematische Operationen

```
println(10 + 3)    // 13    Addition
println(10 - 3)    // 7     Subtraktion
println(10 * 3)    // 30    Multiplikation
println(10 / 3)    // 3     Division (Int!)
println(10 % 3)    // 1     Modulo (Rest)
```

Modulo `%` = der Rest

15 % 5 = 0 → 15 teilbar durch 5

16 % 5 = 1 → Rest 1

Perfekt um zu prüfen:

"Ist eine Zahl durch X teilbar?"

```
zahl % X == 0
```

Vergleiche → true / false

```
println(6 > 7)    // false
println(6 < 7)    // true
println(6 == 6)   // true
println(6 != 7)   // true
```

Operator	Bedeutung
==	ist gleich
!=	ist ungleich
>	größer als
<	kleiner als
>=	größer oder gleich
<=	kleiner oder gleich

for-Loop — Schleifen

Wiederholt Code eine bestimmte Anzahl von Malen:

```
for (i in 1..5) {  
    println(i)  
}  
// → 1 2 3 4 5
```

Syntax

Ergebnis

1..5

1, 2, 3, 4, 5

1 until 5

1, 2, 3, 4 (ohne 5)

5 downTo 1

5, 4, 3, 2, 1

1..10 step 2

1, 3, 5, 7, 9

if / else — Entscheidungen

```
val alter = 16

if (alter >= 18) {
    println("Volljährig!")
} else {
    println("Noch nicht 18.")
}
```

Mehrere Fälle mit else if :

```
if (x == 120) {
    println("YAAAY!")
} else if (x == 5050) {
    println("So you've heard of GAUSS :)")
} else if (x == 6) {
    println("You are just getting started!")
} else {
    println("N0000!")
}
```



Cheat Sheet

Grundlagen — bleibt offen!

Kotlin · Cheat Sheet — Grundlagen

Kurzreferenz zu allen Konzepten aus Teil 2

FUNKTION — BESTANDTEILE

```
fun gruss(name: String): String {  
    return "Hallo $name"  
}
```

TEIL	BEDEUTUNG
fun	Schlüsselwort
gruss	Name
(name: String)	Parameter : Typ
: String	Rückgabotyp
{ ... }	Rumpf

FUNKTION AUFRUFEN

```
fun main() {  
    val text = gruss("Tom") // Rückgabe speichern  
    println(text)  
}
```

AUSGABE

```
println("Text") // mit Umbruch  
print("...")    // ohne
```

VARIABLEN

```
val name = "Mia" // fest  
var punkte = 0    // änderbar
```

Wenn möglich `val`, sonst `var`.

DATENTYPEN

TYP	BEISPIEL
Int	42
Double	3.14
String	"Hallo"
Boolean	true

TYP-INFERENZ · TYP EXPLIZIT

```
val a = 16 // Int erkannt  
val pi: Double = 3.14 // explizit
```

STRING TEMPLATES

```
"Hallo $name"  
"Summe: ${a + b}"
```

MATHE

```
+ - * / // Grundrechnen  
% // Rest (Modulo)  
10 + 3 // → 13  
10 / 3 // → 3 (Int-Division)  
10.0 / 3 // → 3.33... (Double)  
15 % 5 // → 0 (5 teilt 15)  
10 % 3 // → 1 (Rest)
```

VERGLEICHE → BOOLEAN

```
== != // gleich / ungleich  
< <= > >= // kleiner / größer
```

BEDINGUNG — IF / ELSE

```
if (bedingung) {  
    ...  
} else if (bedingung) {  
    ...  
} else {  
    ...  
}  
  
if (zahl % 2 == 0) {  
    println("Zahl ist gerade")  
}
```

SCHLEIFEN — FOR & WHILE

```
for (i in 1..5) { ... } // 1..5  
while (i < 10) { i++ } // solange wahr
```



Live Demo

Wir bauen zusammen ein kleines Programm!

👉 play.kotlinlang.org



Live Demo — Erste Schritte

1. Ausgabe & Variablen

```
fun main() {  
    var x = 0  
    println(x)  
}
```

2. Vergleiche

```
println(3 > 7)    // false  
println(3 < 7)    // true
```

3. Mathe

```
println(6 / 7)    // 0 (Int-Division!)  
println(6f / 7f)  // 0.857... (Float)  
println(16 % 7)   // 2
```




Live Demo — Loops & Bedingungen

4. For-Loop → Variable hochzählen

```
var x = 0
for (i in 1..3) {
    x = x + i
}
println(x)    // → 6
```

5. if / else if / else

```
if (x == 120) {
    println("YAAAY!")
} else if (x == 5050) {
    println("So you've heard of GAUSS :)")
} else if (x == 6) {
    println("You are just getting started!")
} else {
    println("NOOOO!")
}
```



Aufgabe an die Klasse: Was muss der Loop sein, damit `x = 120` rauskommt?

 Jetzt seid ihr dran!

Experimentiert mit `println`, Variablen, Schleifen, `if / else`

 play.kotlinlang.org

Kaputt machen erlaubt — aus Fehlern lernt man am meisten 



Challenge: FizzBuzz

Könnst ihr dieses Rätsel knacken?

Das Cheat Sheet bleibt offen — nutzt es!



Aufgabe 1 — Zahlen ausgeben

Schreibe ein Programm, das die **Zahlen von 1 bis 100** untereinander ausgibt.

Erwartete Ausgabe:

```
1
2
3
4
...
99
100
```



Tipp: `for` -Loop + `println`



Lösung

```
fun main() {
    for (i in 1..100) {
        println(i)
    }
}
```

`1..100` erzeugt eine Range von 1 bis einschließlich 100.

Aufgabe 2 — Fizz!

Manche Zahlen sollen **übersetzt** werden:

Für **Vielfache von 3** wird `"Fizz"` ausgegeben.

Erwartete Ausgabe:

```
1
2
Fizz
4
5
Fizz
7
...
```

💡 Tipp: `if` + Modulo (`%`)

→ Wie prüft man "ist durch 3 teilbar"?

Lösung

```
fun main() {
    for (i in 1..100) {
        if (i % 3 == 0) {
            println("Fizz")
        } else {
            println(i)
        }
    }
}
```



Aufgabe 3 — Buzz!

Zusätzlich: "Buzz" bei Vielfachen von 5.

Erwartete Ausgabe:

```
1
2
Fizz
4
Buzz
Fizz
7
...
```



Tipp: `else if` hinzufügen



Lösung

```
fun main() {
    for (i in 1..100) {
        if (i % 3 == 0) {
            println("Fizz")
        } else if (i % 5 == 0) {
            println("Buzz")
        } else {
            println(i)
        }
    }
}
```

Aufgabe 4 — FizzBuzz!

Zusätzlich: "FizzBuzz" bei Zahlen, die **sowohl** durch 3 als auch durch 5 teilbar sind.

Erwartete Ausgabe:

```
...  
13  
14  
FizzBuzz  
16  
...
```

⚠ Die **Reihenfolge** der `if`-Bedingungen ist entscheidend!

Welche muss **zuerst** geprüft werden?

Lösung

```
fun main() {  
    for (i in 1..100) {  
        if (i % 15 == 0) {           // zuerst!  
            println("FizzBuzz")  
        } else if (i % 3 == 0) {  
            println("Fizz")  
        } else if (i % 5 == 0) {  
            println("Buzz")  
        } else {  
            println(i)  
        }  
    }  
}
```

Eine Zahl die durch 3 **und** 5 teilbar ist, ist auch durch 15 teilbar.



Teil 3

Objektorientierte Programmierung

Warum Objektorientierung?

Bisher: Variablen und Funktionen — alles einzeln.

Problem: Was wenn wir mehrere Haustiere beschreiben wollen?

```
// So? Wird schnell unübersichtlich...
```

```
var pet1Name = "Fluffy"
```

```
var pet1Hunger = 5
```

```
var pet1Happiness = 5
```

```
var pet2Name = "Rex"
```

```
var pet2Hunger = 3
```

```
var pet2Happiness = 8
```

Besser: Alles was zusammengehört in einen **Bauplan** packen → **Klasse!**

Klassen = Baupläne

Eine **Klasse** beschreibt, wie ein Ding aufgebaut ist.

Ein **Objekt** ist ein konkretes Exemplar nach diesem Bauplan.

Klasse "Hund"

Objekte

name	→	Rex, Bella, Lucky
alter	→	3, 7, 1
hunger	→	5, 2, 8
bellen()	→	"Wuff!", "Wuff!", "Wuff!"
füttern()		

Ein Bauplan, **beliebig viele** Objekte.

Was steckt in einer Klasse?

Eine Klasse hat zwei Arten von „Inhalt“:

1. Eigenschaften (Properties)

→ Wie das Ding **ist** (Zustand)

```
name, alter, hunger
```

2. Methoden

→ Was das Ding **tun kann** (Verhalten)

```
bellen(), füttern()
```

Im echten Leben 🐕

Hund Rex

In der Klasse

heißt Rex

```
name = "Rex"
```

ist 3 Jahre

```
alter = 3
```

kann bellen

```
bellen()
```

kann gefüttert werden

```
füttern()
```

👉 Daten und Funktionen, die zusammengehören, leben im **selben Objekt**.

Klasse in Kotlin

```
class Hund(val name: String) {  
    var alter: Int = 3  
    var hunger: Int = 5  
  
    fun bellen() {  
        println("$name sagt: Wuff!")  
    }  
}
```

Bestandteil

Was ist das?

class Hund

Name des Bauplans

(val name:
String)

Parameter beim
Erstellen

var alter: Int =
3

Eigenschaft (Property)

fun bellen()

Methode (Funktion drin)

Objekte erstellen & verwenden

```
fun main() {  
    val rex = Hund("Rex")      // Objekt erstellen  
    val bella = Hund("Bella")  // noch eins  
  
    rex.bellen()                // Rex sagt: Wuff!  
    bella.bellen()              // Bella sagt: Wuff!  
  
    println(rex.hunger)         // 5  
    rex.hunger = 3              // Property ändern  
}
```

Der **Punkt** `.` ist der Zugang:

<code>objekt.eigenschaft</code>	→ Wert lesen/ändern
<code>objekt.methode()</code>	→ Funktion ausführen

Die vier Schritte mit Klassen

1 Definieren — den Bauplan schreiben

```
class Hund(val name: String) {  
    var hunger: Int = 5  
}
```

2 Erstellen — ein Objekt nach dem Bauplan

```
val rex = Hund("Rex")
```

3 Lesen / Ändern — Properties über .

```
println(rex.hunger) // lesen  
rex.hunger = 3      // ändern
```

4 Aufrufen — Methoden ausführen

```
rex.bellen()
```

👉 Mehr braucht es **nicht**, um ein digitales Haustier zu bauen.



Datenstrukturen

Mehrere Werte zusammen verwalten

Listen — viele Werte in Reihenfolge

Bisher: eine Variable → ein Wert. Aber was, wenn wir viele brauchen?

```
val frucht1 = "Apfel"
val frucht2 = "Banane"
val frucht3 = "Kirsche"
// ... wird schnell unübersichtlich
```

Eine **Liste** speichert beliebig viele Werte unter **einem** Namen:

```
val fruechte = listOf("Apfel", "Banane", "Kirsche")

println(fruechte[0])    // → Apfel
println(fruechte.size)  // → 3

for (f in fruechte) {
    println(f)
}
```

Änderbare Liste — Dinge dazupacken & entfernen:

```
val zahlen = mutableListOf(1, 2, 3)
zahlen.add(4)           // → [1,2,3,4]
zahlen.remove(2)        // → [1,3,4]
```

Form

Was?

listOf(...)

fest (immutable)

mutableListOf(...)

änderbar

liste[i]

Wert an Stelle `i`

liste.size

Anzahl Elemente

Maps — Werte mit Schlüssel

Eine Liste ist sortiert nach **Position**. Eine **Map** ordnet jedem **Schlüssel** einen **Wert** zu — wie ein Wörterbuch.

```
val hauptstadt = mapOf(  
    "Deutschland" to "Berlin",  
    "Frankreich"  to "Paris",  
    "Italien"     to "Rom"  
)  
  
println(hauptstadt["Frankreich"]) // → Paris  
println(hauptstadt.size)          // → 3
```

Schlüssel (Wort) → Wert (Bedeutung).

Änderbare Map — Einträge ändern & ergänzen:

```
val alter = mutableMapOf("Anna" to 16)  
alter["Anna"] = 17           // ändern  
alter["Ben"]   = 15          // neu
```

Form	Was?
<code>mapOf("k" to v)</code>	fest
<code>mutableMapOf(...)</code>	änderbar
<code>map["k"]</code>	Wert zum Schlüssel
<code>map["k"] = v</code>	setzen / ändern



Cheat Sheet

OOP + Datenstrukturen — bleibt offen!

Kotlin · Cheat Sheet — OOP + Datenstrukturen

Zusatz zu Teil 2 — Klassen, Objekte, Listen, Maps

IDEE

Eine **Klasse** ist ein Bauplan.

Ein **Objekt** ist ein konkretes Exemplar.

KLASSE	OBJEKT
Bauplan	Exemplar
Hund	rex, bella

- **Properties** — wie das Ding ist (Zustand)
- **Methoden** — was es tun kann (Verhalten)

KLASSE — SKELETT

```
class Name(val p: Typ) {  
    var eigenschaft: Typ = wert  
  
    fun methode() {  
        eigenschaft = eigenschaft + 1  
    }  
}
```

TEIL	BEDEUTUNG
class Name	Bauplan-Name
(val p: Typ)	Konstruktor-Parameter
var ... = ...	Property (Zustand)
fun ...() { ... }	Methode (Verhalten)

OBJEKT ERSTELLEN

```
val a = Name("...")  
val b = Name("...")    // beliebig viele
```

ZUGRIFF MIT .

```
objekt.eigenschaft      // lesen  
objekt.eigenschaft = ... // ändern  
objekt.methode()        // ausführen
```

Der Punkt `.` ist der Zugang zum Objekt.

METHODE MIT RÜCKGABE

```
fun beschreibung(): String {  
    return "..."  
}
```

Rückgabebetyp hinter `:` `return` liefert den Wert.

LISTE — VIELE WERTE IN REIHENFOLGE

```
val xs = mutableListOf("a", "b")  
xs.add("c")           // → [a, b, c]  
xs[0]                 // → "a"  
xs.size               // → 3  
for (x in xs) { ... }
```

`listOf(...)` fest · `mutableListOf(...)` änderbar.

MAP — SCHLÜSSEL → WERT

```
val m = mutableMapOf("Anna" to 16)  
m["Anna"] = 17      // ändern  
m["Ben"] = 15       // neu  
m["Anna"]           // → 17  
m.size              // → 2
```

Wie ein Wörterbuch: Schlüssel → Wert.



Aufgabe: Steckbrief

Erstellt euren eigenen digitalen Steckbrief!

Steckbrief — Aufgabe

Baut eure **eigene Klasse** Steckbrief im Playground.

Sie soll **euch beschreiben**:

- Name
- Alter
- Hobby
- ... mindestens **2 weitere** Eigenschaften, die zu euch passen

Außerdem soll der Steckbrief sich selbst **vorstellen können** — z.B. mit einer Methode `print()`, die alle Daten auf der Konsole ausgibt.

Testet eure Klasse, indem ihr ein Objekt mit euren Daten erstellt und `print()` aufruft.

Ziel-Ausgabe (Beispiel)

```
Hallo, ich heiße Alex.  
Ich bin 16 Jahre alt.  
Mein Hobby ist Zocken.  
Mein Lieblingsessen ist Pizza.  
Mein Lieblingsfach ist Informatik.
```



Bonus: Erstellt ein **zweites Objekt** für euren Sitznachbarn — dieselbe Klasse, andere Daten.



Steckbrief — Beispiel-Lösung

```
class Steckbrief(val name: String) {  
    var alter: Int = 0  
    var hobby: String = ""  
    var lieblingsessen: String = ""  
    var Lieblingsfach: String = ""  
  
    fun print() {  
        println("Hallo, ich heiße $name.")  
        println("Ich bin $alter Jahre alt.")  
        println("Mein Hobby ist $hobby.")  
        println("Mein Lieblingsessen ist $lieblingsessen.")  
        println("Mein Lieblingsfach ist $Lieblingsfach.")  
    }  
}
```

```
fun main() {  
    val ich = Steckbrief("Alex")  
    ich.alter = 16  
    ich.hobby = "Zocken"  
    ich.lieblingsessen = "Pizza"  
    ich.lieblingsfach = "Informatik"  
    ich.print()  
  
    println("---")  
  
    val nachbar = Steckbrief("Sam")  
    nachbar.alter = 15  
    nachbar.hobby = "Fußball"  
    nachbar.lieblingsessen = "Sushi"  
    nachbar.lieblingsfach = "Sport"  
    nachbar.print()  
}
```



Cheat Sheet

Komplett — bleibt offen für die Pet-Kata!

⊖ BASICS

PROGRAMM-SKELETT & AUSGABE

```
fun main() {
    println("Text") // mit Umbruch
    print("...")    // ohne
    val x = readln() // Eingabe
}
```

VARIABLEN

```
val name = "Mia" // fest
var punkte = 0   // änderbar
```

Wenn möglich `val`, sonst `var`.

DATENTYPEN · INFERENZ

TYP	BEISPIEL
Int	42
Double	3.14
String	"Hallo"
Boolean	true

```
val a = 16 // Int erkannt
val pi: Double = 3.14 // explizit
```

STRING TEMPLATES

```
"Hallo $name"
"Summe: ${a + b}"
```

⊖ FUNKTIONEN

BESTANDTEILE

```
fun gruss(name: String): String {
    return "Hallo $name"
}
```

TEIL	BEDEUTUNG
fun	Schlüsselwort
gruss	Name
(name: String)	Parameter: Typ
: String	Rückgabetyt
{ ... }	Rumpf

AUFRUFEN

```
fun main() {
    val text = gruss("Tom")
    println(text)
}
```

⊖ MATHE & VERGLEICHE

MATHE

```
+ - * / // Grundrechnen
% // Rest (Modulo)
10 + 3 // → 13
10 / 3 // → 3 (Int-Division)
10.0 / 3 // → 3.33... (Double)
15 % 5 // → 0 (5 teilt 15)
10 % 3 // → 1 (Rest)
```

VERGLEICHE → BOOLEAN

```
== != // gleich / ungleich
< <= > >= // kleiner / größer
```

⊖ KONTROLLFLUSS

BEDINGUNG — IF / ELSE

```
if (bedingung) {
    ...
} else if (bedingung) {
    ...
} else {
    ...
}

if (zahl % 2 == 0) {
    println("Zahl ist gerade")
}
```

SCHLEIFEN — FOR & WHILE

```
for (i in 1..5) {
    println(i)
}

while (i < 10) {
    i++
}
```

WERT BEGRENZEN MIT IF

```
if (punkte > 100) {
    punkte = 100
}

if (punkte < 0) {
    punkte = 0
}
```

⊖ OOP — SYNTAX

KLASSE — SKELETT

```
class Name(val p: Typ) {
    var eigenschaft: Typ = wert

    fun methode() {
        eigenschaft = eigenschaft + 1
    }

    fun beschreibung(): String {
        return "..."
    }
}
```

TEIL	BEDEUTUNG
class Name	Bauplan-Name
(val p: Typ)	Konstruktor-Parameter
var _ = _	Property (Zustand)
fun _() { ... }	Methode (Verhalten)
: String + return	Rückgabewert

OBJEKT ERSTELLEN & NUTZEN

```
val a = Name("...") // erstellen
val b = Name("...") // beliebig viele

a.eigenschaft = ... // ändern
println(a.eigenschaft) // lesen
a.methode() // aufrufen
val t = a.beschreibung() // Rückgabe
```

Der Punkt `.` ist der Zugang zum Objekt. Jedes Objekt hat seinen eigenen Zustand.

⊖ DATENSTRUKTUREN

LISTE — VIELE WERTE IN REIHENFOLGE

```
val obst = mutableListOf("Apfel", "Banane")
obst.add("Kirsche") // → [Apfel, Banane, Kirsche]
obst[0] // → "Apfel"
obst.size // → 3
for (o in obst) { ... }
```

MAP — SCHLÜSSEL → WERT

```
val noten = mutableMapOf("Mathe" to 2)
noten["Mathe"] = 1 // ändern
noten["Informatik"] = 1 // neu
noten["Mathe"] // → 1
noten.size // → 2
```




Übergang: Digital Pet

Was bauen wir jetzt?

Ein virtuelles Haustier

- Es hat **Hunger** und **Glück**
- Man kann es **füttern** und mit ihm **spielen**
- Man kann seinen **Status** abfragen
- Am Ende steuert man es über ein **Menü im Terminal**

Ihr baut das Schritt für Schritt — Aufgabe für Aufgabe.

Grundstruktur

```
class Pet(val name: String) {  
    var hunger: Int = 5  
    // 0 = satt, 10 = am Verhungern  
  
    var happiness: Int = 5  
    // 0 = traurig, 10 = überglücklich  
}
```



Coding Kata: Digital Pet



Baut Schritt für Schritt euer eigenes Haustier!

Alles im **Playground** — bis Aufgabe 5, da wechseln wir zu IntelliJ.



Pet — Aufgabe 1

Klasse erstellen

Erstelle die Klasse `Pet`, die beim Erstellen einen **Namen** bekommt.

Das Tier startet mit zwei Werten:

- `hunger: 5` (0 = satt, 10 = am Verhungern)
- `happiness: 5` (0 = todtraurig, 10 = überglücklich)



Lösung

```
class Pet(val name: String) {  
    var hunger: Int = 5  
    var happiness: Int = 5  
}  
  
fun main() {  
    val pet = Pet("Fluffy")  
    println("Name: ${pet.name}")  
    println("Hunger: ${pet.hunger}")  
    println("Happiness: ${pet.happiness}")  
}
```

Pet — Aufgabe 2

Haustier füttern

Füge der Klasse `Pet` eine Methode `feed()` hinzu.

Regeln

- `hunger` **sinkt um 2** (Futter macht satt)
- `happiness` **steigt um 1** (Essen macht glücklich)

Kein Wert darf unter 0 oder über 10 gehen!

💡 Tipp: Mit `if` begrenzen, damit nichts unter 0 oder über 10 geht:

```
if (hunger > 10) hunger = 10
if (hunger < 0)  hunger = 0
```

Lösung

```
class Pet(val name: String) {
    var hunger: Int = 5
    var happiness: Int = 5

    fun feed() {
        hunger = hunger - 2
        happiness = happiness + 1

        if (hunger < 0) hunger = 0
        if (happiness > 10) happiness = 10

        println("$name wurde gefüttert! 🍖")
    }
}
```

Pet — Aufgabe 3 (10 min)

Mit Haustier spielen

Füge eine Methode `play()` hinzu.

Regeln

- `happiness` **steigt um 2** (Spielen macht Spaß!)
- `hunger` **steigt um 1** (Spielen macht hungrig)

Weiterhin gilt: Kein Wert darf unter 0 oder über 10 gehen!

Lösung

```
fun play() {  
    happiness = happiness + 2  
    hunger = hunger + 1  
  
    if (happiness > 10) happiness = 10  
    if (hunger > 10) hunger = 10  
  
    println("$name hat gespielt! )  
}
```

Pet — Aufgabe 4 (30 min)

Status anzeigen

Erstelle eine Methode `getStatus()`, die einen **beschreibenden Text** des Haustiers zurückgibt.

Dabei soll der Text widerspiegeln, **wie glücklich und hungrig** das Haustier gerade ist.

Der Text könnte zum Beispiel lauten

`hunger = 4` und `happiness = 10`:

→ „*Fluffy ist etwas hungrig und super glücklich 😊*“

`hunger = 2` und `happiness = 5`:

→ „*Fluffy ist voll gefressen und ganz zufrieden*“

⚠ Ab hier wechseln wir zu IntelliJ

Aufgabe 5 ist interaktiv — dafür brauchen wir ein richtiges Terminal

Pet — Aufgabe 5 (60 min)

Interaktiv mit dem Haustier spielen

Wenn du das Programm startest, wird zunächst der **Name des Haustieres** abgefragt.

Anschließend kann der Benutzer **beliebig oft** zwischen den folgenden Optionen wählen, indem er die entsprechende Nummer im Terminal eingibt:

- 1 → Füttern
- 2 → Spielen
- 3 → Status ansehen
- 0 → Beenden

Nach jeder Aktion wird dem Nutzer ein Text angezeigt, der beschreibt, welche Optionen ihm

Lösung

```
fun main() {
    println("=== DIGITALES HAUSTIER ===")
    print("Wie soll dein Haustier heißen? ")
    val name = readln()
    val pet = Pet(name)

    var running = true
    while (running) {
        println("\n--- Runde ---")
        println(pet.getStatus())
        println("Was möchtest du tun?")
        println(" 1 → Füttern 🍖")
        println(" 2 → Spielen 🎱")
        println(" 3 → Status ansehen 📊")
        println(" 0 → Beenden 🛑")
        print("> ")

        val eingabe = readln()
        if (eingabe == "1") {
            pet.feed()
        } else if (eingabe == "2") {
            pet.play()
        } else if (eingabe == "3") {
            println(pet.getStatus())
        } else if (eingabe == "0") {
```



Geschafft!

Ihr habt heute gelernt:

- Variablen, Typen, Templates
- Schleifen & Bedingungen
- Klassen & Objekte
- FizzBuzz gelöst
- Eigenen Steckbrief gebaut
- Digitales Haustier programmiert

Wer weiterprogrammieren will → play.kotlinlang.org

Danke fürs Mitmachen! A small rocket icon with a flame, symbolizing speed, progress, or a final launch.